

Unified Framework for Adaptive Power-Law Spectral Probing and Nonlinear-Response Detection

Adaptive Sensor Attention Through Closed-Loop Spectral Allocation

Lu Semita

EmergenceByDesign

July 2026

A formal record of ongoing engineering research and development intended to document the evolution of ideas prior to comprehensive literature review and future experimental validation. The present work does not assert ownership, discovery, invention, or priority over concepts that may independently exist within prior or contemporary research. It serves as a dated record of development work and its associated implementations.

Contents

Abstract	1
1. Scope and central hypothesis	1
1.1 Power versus amplitude	1
1.2 What the framework does not establish	2
2. Adaptive sensor attention	2
2.1 Practical use cases	2
2.2 Structural correspondence with biological sensing	3
3. Engineering foundations	3
2.1 Crest factor and PAPR management	3
2.2 Regularized transducer compensation	4
2.3 Feedback updates	4
2.4 Nonlinear response and subharmonics	4
2.5 Sampling and cyclic waveform constraints	4
2.6 Deployment qualification	5
4. Feedback objectives and computational architecture	5
4.1 Computational cost and real-time deployment	6
5. Reference implementation: shared utilities	6
6. Platform 1: ultrasonic NDT	8
6.1 Interpretation	8
6.2 Python implementation	8
7. Platform 2: marine acoustic probe	13
7.1 Interpretation	13
7.2 Python implementation	13
7.3 Recommended measurement separation	16
8. Platform 3: LiDAR intensity-modulation multisine	16
8.1 Interpretation	16
8.2 Python implementation	16
9. Fully executable consolidated implementation	18
10. Validation strategy	19
10.1 Numerical validation	19
10.2 Electrical and electroacoustic validation	19
10.3 NDT nonlinear validation	19
10.4 Sonar validation	19
10.5 LiDAR validation	19
11. Demonstration figures and performance interpretation	19
11.1 Interpreting improvement	20
12. Conclusions	20
References	22

Unified Framework for Adaptive Power-Law Spectral Probing and Nonlinear-Response Detection

Adaptive Sensor Attention Through Closed-Loop Spectral Allocation

Lu Semita

EmergenceByDesign

July 2026

A formal record of ongoing engineering research and development intended to document the evolution of ideas prior to comprehensive literature review and future experimental validation. The present work does not assert ownership, discovery, invention, or priority over concepts that may independently exist within prior or contemporary research. It serves as a dated record of development work and its associated implementations.

Abstract

This document develops a hardware-aware framework for adaptive active sensing across ultrasonic nondestructive testing (NDT), marine acoustics, and amplitude-modulated LiDAR. The framework does **not** assume that every propagation channel possesses a universal $1/f$ response. Instead, it treats a power-law spectral weighting,

$$S_x(f) \propto f^{-\alpha},$$

as a tunable probe family whose exponent α must be selected or learned for the instrument, medium, target, and measurement objective.

The implementations distinguish power spectral density from Fourier amplitude, use regularized complex inverse filtering rather than unconstrained division, constrain excitation to calibrated passbands, quantify crest factor consistently, and update spectral weights with bounded, smoothed feedback. The nonlinear-ultrasonic section also distinguishes **transmitting a lower-frequency companion band** from **detecting subharmonics generated by the specimen**. The LiDAR implementation generates a cyclic intensity-modulation multisine; it does not claim to implement optical FMCW ranging.

The code provides simulation- and waveform-design reference implementations. It does not replace instrument-specific safety limits, calibration, regulatory review, impedance matching, anti-alias filtering, or real-time firmware.

1. Scope and central hypothesis

Traditional active sensing often uses single pulses, linear or logarithmic chirps, pseudorandom sequences, or multisines. A power-law-weighted probe can complement these methods when prior measurements suggest that equal energy per linear-frequency bin wastes dynamic range or under-excites informative regions.

The proposed adaptive loop is:

1. calibrate the transmitter and receiver chains;
2. choose a physically valid passband;
3. synthesize a probe with a target power-law spectrum;
4. constrain crest factor, RMS level, peak level, and spectral gain;
5. transmit and acquire a synchronized response;
6. estimate channel or feature statistics only where the transmitted spectrum has adequate signal-to-noise ratio;
7. update the spectral envelope using bounded, regularized, smoothed weights;
8. stop according to a defined objective, such as spectral flatness, parameter uncertainty, repeatability, or information gain.

This framework supports a **family** of spectra. White power corresponds to $\alpha = 0$; pink-like power corresponds to $\alpha = 1$; other exponents can model or compensate measured attenuation.

1.1 Power versus amplitude

If the desired one-sided power spectral density follows

$$S_x(f) \propto f^{-\alpha},$$

then the Fourier **amplitude** should scale approximately as

$$|X(f)| \propto f^{-\alpha/2}.$$

Therefore, a pink-power target ($S_x \propto 1/f$) uses an amplitude envelope proportional to $1/\sqrt{f}$, not $1/f$. An amplitude proportional to $1/f$ instead produces power proportional to $1/f^2$.

1.2 What the framework does not establish

The framework does not establish that all physical media exhibit a $1/f$ attenuation law. Acoustic, elastic, and optical attenuation can follow frequency-linear, quadratic, fractional-power, exponential, resonant, or piecewise behavior. The exponent and weighting objective must therefore remain empirical and frame-specific.

2. Adaptive sensor attention

The framework can be understood most directly as an **attention mechanism for active sensors**. A conventional sensor generally follows a predetermined schedule: emit a pulse, chirp, or multisine; record the return; and repeat the same interrogation regardless of what the previous measurement revealed. That approach remains appropriate for routine screening because it minimizes computation, calibration burden, and acquisition time.

Adaptive probing adds a second mode of operation. It treats time, energy, bandwidth, and computation as a finite sensing budget, then reallocates that budget according to feedback from earlier measurements. The sensor does not necessarily become more powerful. It becomes more selective about where additional effort goes.

The architecture follows a closed loop:

1. perform a broad initial probe;
2. identify reliable or anomalous portions of the response;
3. alter the next probe according to a stated objective;
4. measure again;
5. continue until improvement slows, uncertainty reaches a threshold, or the available sensing budget expires.

This additional attention has an up-front cost. It requires calibration, synchronized feedback, more computation, and sometimes multiple acquisitions. For a simple scan of a well-characterized object, that cost may provide little value. The method therefore appears best suited to outlier cases, difficult environments, and diagnostic work in which an ambiguous initial measurement justifies focused follow-up.

2.1 Practical use cases

Ultrasonic inspection of a suspicious weld or aircraft component. A routine pulse-echo scan can first identify an unusual region. Rather than rescanning the entire component uniformly, the adaptive stage can concentrate spectral effort around frequencies that produce stable defect-sensitive responses. In lay terms, the system stops tapping every part of the object in exactly the same way and begins listening for which kinds of taps make the suspicious region answer differently.

Medical or laboratory ultrasound. An initial image may reveal an ambiguous tissue boundary or material interface. Subsequent probes can emphasize bands that improve contrast or reduce uncertainty while suppressing bands that mainly produce clutter. This resembles autofocus: the hardware remains the same, but the interrogation becomes progressively better matched to the observed scene.

Marine sonar and sub-bottom profiling. Water, sediment, bubbles, and platform motion can make some frequencies far more useful than others. A first ping can estimate which regions of the spectrum survive the channel and return useful structure. Later pings can allocate more energy to those regions or compare separated companion bands without simply raising total source level.

Subsurface mapping and anomaly investigation. A broad survey can locate a buried object, void, or geological transition. Adaptive probing can then spend additional time on the anomalous region, using the response itself to choose more informative spectral weights.

LiDAR through difficult atmospheric conditions. Fog, dust, and material reflectance can alter the quality of modulation returns. A calibrated intensity-modulation system can compare candidate

modulation tones and favor those that preserve useful contrast, subject to optical bandwidth and safety limits.

Across these examples, the expected benefit does not take the form of a universal range multiplier. The benefit appears as better allocation of the existing measurement budget: improved defect SNR, reduced parameter uncertainty, faster convergence on an ambiguous target, or fewer unnecessary high-cost measurements elsewhere.

2.2 Structural correspondence with biological sensing

The biological analogies discussed here were noticed **after** the engineering framework had been formulated. They did not inspire the design and do not establish its validity. They instead reveal a structural correspondence: systems facing finite sensing resources, noisy environments, incomplete information, and sequential decisions often develop a pattern of broad detection followed by selective attention.

Direct analogs include:

- **Foveated vision and saccades.** Peripheral vision performs broad surveillance, while rapid eye movements repeatedly bring selected regions onto the high-resolution fovea.
- **Auditory attention.** Neural processing can increase sensitivity to a behaviorally relevant sound stream while suppressing competing signals.
- **Echolocation.** Bats and other echolocating animals vary call timing, duration, and spectral content as distance, clutter, and target behavior change.
- **Chemotaxis.** Bacteria and mobile cells bias motion according to changing chemical concentrations, repeatedly sampling whether conditions improve or worsen.
- **Olfactory and gustatory adaptation.** Receptor sensitivity and downstream weighting change with exposure, novelty, concentration, and physiological state.
- **Immune surveillance.** Molecular recognition and cellular response become selectively amplified when patterns match learned or evolutionarily conserved danger signals.

Biochemical sensory systems do not attend neutrally. Receptor distributions, signaling gains, thresholds, and adaptation rates embody accumulated evolutionary training, developmental history, and current physiological state. Environmental stimuli receive different weights because some patterns have repeatedly correlated with food, threat, injury, reproduction, homeostasis, or other core imperatives. In learned organisms, experience further modifies those weights.

The engineered framework follows a comparable organization without claiming mechanistic identity. Its initial spectral prior, calibration data, feedback objective, and stopping rules function as explicit weights. They determine what counts as informative, which deviations deserve additional attention, and when the system should stop spending resources. In that sense, the sensor does not merely measure the environment; it measures according to a trained or designed hierarchy of relevance.

The recurring structural pattern is therefore:

Probe -> Observe -> Weight -> Reallocate Attention -> Probe Again

This correspondence suggests a broader interpretation of the work. The waveform generator provides one implementation layer, but the more general contribution lies in a closed-loop architecture for allocating physical sensing effort according to evidence, training data, and operational imperatives.

3. Engineering foundations

2.1 Crest factor and PAPR management

For a real signal $x[n]$, crest factor is

$$CF = 20 \log_{10} \left(\frac{\max_n |x[n]|}{\text{RMS}(x)} \right).$$

For deterministic multisines, quadratic phase schedules associated with Newman/Schroeder-style constructions often reduce crest factor when tones occupy regularly spaced frequencies. They do not guarantee a global optimum for arbitrary tone placement and arbitrary amplitudes. The implementations therefore:

- use FFT-bin-centered tones;
- apply a quadratic phase schedule as an initial low-crest-factor construction;
- measure the achieved crest factor;
- normalize by RMS and then enforce a peak limit.

2.2 Regularized transducer compensation

Directly dividing by $H_{\text{tx}}(f)$ can create unbounded gain near response nulls. A regularized complex inverse is safer:

$$G_{\text{inv}}(f) = \frac{H_{\text{tx}}^*(f)}{|H_{\text{tx}}(f)|^2 + \lambda},$$

followed by a maximum-gain clamp and an explicit calibrated passband mask. This form preserves phase information when a complex transfer function is available.

2.3 Feedback updates

The raw ratio $|Y|/|X|$ becomes unreliable where $|X|$ approaches the noise floor. A stable update should:

- ignore bins with insufficient transmitted energy or coherence;
- smooth the estimated response;
- use a bounded target-to-observed ratio;
- blend new and previous weights with a learning rate;
- clamp the result to a permitted range;
- normalize only within the active passband.

The example uses spectral-flattening feedback. A production system may instead optimize Fisher information, mutual information, classification margin, defect-feature SNR, or posterior uncertainty.

2.4 Nonlinear response and subharmonics

A subharmonic response is an output component at f_0/n generated by nonlinear dynamics when the specimen is driven near f_0 . Merely transmitting a second chirp at $f_0/2$ does not demonstrate subharmonic generation; it creates an additional low-frequency probe.

Accordingly:

- the NDT module provides a detector for output energy near expected subharmonic bands after fundamental excitation;
- the sonar module calls its lower-frequency component a **companion band**, not a detected subharmonic;
- nonlinear claims require controls for electronics, fixtures, couplant, transducer nonlinearity, and baseline specimens.

2.5 Sampling and cyclic waveform constraints

Every generated frequency must remain below Nyquist and should normally retain transition margin for reconstruction filters. For cyclic DAC playback, tones should fall on FFT bins to avoid endpoint discontinuity and spectral leakage. The code enforces a configurable safety fraction of the sampling rate.

2.6 Deployment qualification

The Python implementations require NumPy, and the sonar example requires SciPy. They suit desktop Python, many Linux single-board computers, and instrument hosts. Standard MicroPython installations generally do not provide NumPy/SciPy-compatible FFT and signal-processing stacks; embedded deployment requires porting the generated tables or algorithms to CMSIS-DSP, vendor FPGA logic, C/C++, or another supported numerical runtime.

4. Feedback objectives and computational architecture

Spectral flattening provides a transparent default objective, but the architecture should not hard-code it as the only measure of improvement. Different sensing tasks require different definitions of attention. A diagnostic system may maximize defect-sensitive SNR; a classifier may maximize separation between target classes; an experimental platform may minimize posterior uncertainty; and a resource-constrained instrument may maximize information gained per joule or per unit time.

A minimal extension interface can isolate that decision from waveform synthesis:

```
from abc import ABC, abstractmethod
import numpy as np

class FeedbackObjective(ABC):
    """Convert measured spectra into bounded spectral update weights."""

    @abstractmethod
    def compute_weights(
        self,
        transmitted_fft: np.ndarray,
        received_fft: np.ndarray,
        current_weights: np.ndarray,
        reliable_mask: np.ndarray,
    ) -> np.ndarray:
        raise NotImplementedError

class SpectralFlatteningObjective(FeedbackObjective):
    """Example objective that reduces passband response variation."""

    def compute_weights(
        self,
        transmitted_fft: np.ndarray,
        received_fft: np.ndarray,
        current_weights: np.ndarray,
        reliable_mask: np.ndarray,
    ) -> np.ndarray:
        response = np.abs(received_fft) / np.maximum(
            np.abs(transmitted_fft), 1e-15
        )
        target = np.median(response[reliable_mask])
        proposed = np.ones_like(response)
        proposed[reliable_mask] = target / np.maximum(
            response[reliable_mask], 1e-15
        )
        return proposed
```

This separation permits alternative objectives such as Fisher-information maximization, defect SNR, mutual information, classification margin, Bayesian uncertainty reduction, or application-specific utility functions while leaving the synthesis and hardware-safety layers intact.

4.1 Computational cost and real-time deployment

The dominant operations consist of FFTs, inverse FFTs, and element-wise vector arithmetic. For a buffer containing N samples, the principal cost scales approximately as

$$O(N \log N).$$

Typical desktop processors and instrument hosts can execute these operations comfortably for moderate buffer sizes. Real-time embedded implementations can reduce latency by precomputing frequency masks, phase schedules, calibration inverses, and initial spectral envelopes. CMSIS-DSP, optimized C/C++ FFT libraries, FPGA pipelines, or fixed-point arithmetic may replace the Python reference implementation where deterministic timing matters.

Adaptation need not occur for every emitted sample. Many systems can separate fast playback from slower supervisory updates: hardware repeats a precomputed waveform while a host processor evaluates several acquisitions and prepares the next buffer. This division preserves real-time signal generation while allowing more sophisticated objectives to run at a lower control rate.

5. Reference implementation: shared utilities

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Iterable, Optional, Sequence, Tuple

import numpy as np

def rms(x: np.ndarray) -> float:
    """Root-mean-square magnitude."""
    x = np.asarray(x, dtype=float)
    return float(np.sqrt(np.mean(np.square(x))))

def crest_factor_db(x: np.ndarray, eps: float = 1e-15) -> float:
    """20 log10(peak / RMS), reported in dB."""
    x = np.asarray(x, dtype=float)
    return float(20.0 * np.log10(
        (np.max(np.abs(x)) + eps) / (rms(x) + eps)
    ))

def db_to_amplitude(db: float) -> float:
    """Convert an amplitude ratio in dB to a linear amplitude ratio."""
    return float(10.0 ** (db / 20.0))

def normalize_rms_and_limit_peak(
    x: np.ndarray,
    target_rms: float = 0.20,
    peak_limit: float = 0.95,
    eps: float = 1e-15,
) -> np.ndarray:
    """
    Set RMS where possible, then apply a final peak limit.
    This avoids describing peak-normalized signals as equal-power signals.
    """
```

```

y = np.asarray(x, dtype=float).copy()
current_rms = rms(y)
if current_rms > eps:
    y *= target_rms / current_rms

peak = float(np.max(np.abs(y)))
if peak > peak_limit:
    y *= peak_limit / peak
return y

def power_law_amplitude(
    frequencies: np.ndarray,
    alpha: float,
    f_reference: float,
    active_mask: np.ndarray,
) -> np.ndarray:
    """
    Return Fourier amplitudes for PSD proportional to  $f^{*(-alpha)}$ .

    Since PSD is proportional to amplitude squared, amplitude scales as
     $f^{*(-alpha/2)}$ . DC and inactive bins remain zero.
    """
    f = np.asarray(frequencies, dtype=float)
    mask = np.asarray(active_mask, dtype=bool) & (f > 0.0)
    amplitude = np.zeros_like(f)
    amplitude[mask] = (f[mask] / f_reference) ** (-0.5 * alpha)
    return amplitude

def quadratic_multisine_phases(number_of_tones: int) -> np.ndarray:
    """
    Quadratic phase seed for a regularly spaced multisine.

    This is a low-crest-factor heuristic, not a guarantee of the global
    minimum for arbitrary amplitudes or irregular tone placement.
    """
    if number_of_tones < 1:
        return np.empty(0, dtype=float)
    m = np.arange(number_of_tones, dtype=float)
    return np.pi * m * (m - 1.0) / number_of_tones

def regularized_inverse(
    transfer_function: np.ndarray,
    regularization: float = 1e-3,
    maximum_gain: float = 8.0,
) -> np.ndarray:
    """Complex Tikhonov-style inverse with a gain clamp."""
    h = np.asarray(transfer_function, dtype=complex)
    g = np.conj(h) / (np.abs(h) ** 2 + regularization)
    magnitude = np.abs(g)
    too_large = magnitude > maximum_gain
    g[too_large] *= maximum_gain / magnitude[too_large]
    return g

```

```
def smooth_vector(values: np.ndarray, width: int = 9) -> np.ndarray:
    """Short symmetric moving-average smoother."""
    x = np.asarray(values, dtype=float)
    width = max(1, int(width))
    if width == 1:
        return x.copy()
    kernel = np.ones(width, dtype=float) / width
    return np.convolve(x, kernel, mode="same")
```

6. Platform 1: ultrasonic NDT

6.1 Interpretation

This implementation generates a calibrated, power-law-weighted multisine inside an ultrasonic transducer passband. It supports bounded spectral feedback and includes a separate subharmonic-response detector.

The detector should be applied to a received signal obtained while driving the specimen at a selected fundamental band. A useful experiment also records:

- an electronics-only loopback;
- a transducer/couplant reference;
- an undamaged specimen baseline;
- repeated acquisitions at several drive amplitudes.

A defect-associated nonlinear response should scale and localize differently from these controls.

6.2 Python implementation

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Optional, Sequence, Tuple

import numpy as np

@dataclass
class ProbeMetrics:
    rms: float
    peak: float
    crest_factor_db: float
    active_bins: int

class UltrasonicNDTProbe:
    """
    Hardware-aware ultrasonic probe synthesizer.

    The target power spectrum is proportional to  $f^{*(-\alpha)}$ . The class applies a calibrated passband, a regularized complex inverse, bounded feedback weights, quadratic multisine phases, RMS normalization, and peak limiting.
    """

    def __init__(
        self,
```

```

sample_rate: float = 10e6,
num_samples: int = 4096,
f_lower: float = 0.5e6,
f_upper: float = 2.0e6,
transducer_response: Optional[np.ndarray] = None,
) -> None:
    if sample_rate <= 0 or num_samples < 16:
        raise ValueError("sample_rate must be positive and num_samples >= 16.")
    if not (0 < f_lower < f_upper < sample_rate / 2):
        raise ValueError("Passband must lie strictly below Nyquist.")

    self.fs = float(sample_rate)
    self.N = int(num_samples)
    self.freqs = np.fft.rfftfreq(self.N, d=1.0 / self.fs)
    self.f_lower = float(f_lower)
    self.f_upper = float(f_upper)
    self.passband = (
        (self.freqs >= self.f_lower) &
        (self.freqs <= self.f_upper)
    )

    if transducer_response is None:
        # Demonstration model only. Replace with measured complex H_tx(f).
        center = 1.25e6
        sigma = 0.35e6
        magnitude = np.exp(
            -0.5 * ((self.freqs - center) / sigma) ** 2
        )
        delay_seconds = 0.35e-6
        phase = -2.0 * np.pi * self.freqs * delay_seconds
        self.H_tx = magnitude * np.exp(1j * phase)
    else:
        response = np.asarray(transducer_response, dtype=complex)
        if response.shape != self.freqs.shape:
            raise ValueError(
                "transducer_response must match rFFT frequency shape."
            )
        self.H_tx = response

def synthesize(
    self,
    target_band: Tuple[float, float],
    alpha: float = 1.0,
    target_gain_db: float = 6.0,
    feedback_weights: Optional[np.ndarray] = None,
    inverse_regularization: float = 1e-3,
    maximum_inverse_gain: float = 6.0,
    target_rms: float = 0.20,
    peak_limit: float = 0.95,
) -> tuple[np.ndarray, ProbeMetrics]:
    f0, f1 = map(float, target_band)
    if not (self.f_lower <= f0 < f1 <= self.f_upper):
        raise ValueError("target_band must lie inside the calibrated passband.")

    amplitude = power_law_amplitude(
        self.freqs,

```

```

        alpha=alpha,
        f_reference=np.sqrt(self.f_lower * self.f_upper),
        active_mask=self.passband,
    )

    target_mask = (self.freqs >= f0) & (self.freqs <= f1)
    amplitude[target_mask] *= db_to_amplitude(target_gain_db)

    if feedback_weights is not None:
        weights = np.asarray(feedback_weights, dtype=float)
        if weights.shape != amplitude.shape:
            raise ValueError("feedback_weights has the wrong shape.")
        amplitude *= np.clip(weights, 0.25, 4.0)

    inverse = regularized_inverse(
        self.H_tx,
        regularization=inverse_regularization,
        maximum_gain=maximum_inverse_gain,
    )
    corrected = amplitude.astype(complex) * inverse
    corrected[~self.passband] = 0.0

    active = np.flatnonzero(np.abs(corrected) > 0.0)
    phases = quadratic_multisine_phases(active.size)
    corrected[active] *= np.exp(1j * phases)

    # DC and Nyquist bins must be real for an exactly real irFFT signal.
    corrected[0] = 0.0
    if self.N % 2 == 0:
        corrected[-1] = corrected[-1].real + 0j

    waveform = np.fft.irfft(corrected, n=self.N)
    waveform = normalize_rms_and_limit_peak(
        waveform,
        target_rms=target_rms,
        peak_limit=peak_limit,
    )

    metrics = ProbeMetrics(
        rms=rms(waveform),
        peak=float(np.max(np.abs(waveform))),
        crest_factor_db=crest_factor_db(waveform),
        active_bins=int(active.size),
    )
    return waveform, metrics

def update_feedback(
    self,
    received_signal: np.ndarray,
    transmitted_signal: np.ndarray,
    previous_weights: Optional[np.ndarray] = None,
    learning_rate: float = 0.25,
    min_tx_relative: float = 1e-3,
    weight_bounds: Tuple[float, float] = (0.4, 2.5),
    smoothing_bins: int = 9,
) -> tuple[np.ndarray, dict]:

```

```

"""
Build bounded weights that tend to flatten the measured magnitude
response inside the calibrated passband.

Replace this objective in production when a different information
criterion is desired.
"""

rx = np.asarray(received_signal, dtype=float)
tx = np.asarray(transmitted_signal, dtype=float)
if rx.shape != (self.N,) or tx.shape != (self.N,):
    raise ValueError(f"Signals must each contain {self.N} samples.")

X = np.fft.rfft(tx)
Y = np.fft.rfft(rx)
tx_mag = np.abs(X)
response = np.abs(Y) / np.maximum(tx_mag, 1e-15)

reliable = (
    self.passband &
    (tx_mag >= min_tx_relative * np.max(tx_mag[self.passband]))
)
smoothed = smooth_vector(response, width=smoothing_bins)

target_level = np.median(smoothed[reliable])
desired = np.ones_like(smoothed)
desired[reliable] = target_level / np.maximum(smoothed[reliable], 1e-15)
desired = np.clip(desired, *weight_bounds)
desired[~self.passband] = 0.0

if previous_weights is None:
    previous = np.ones_like(desired)
    previous[~self.passband] = 0.0
else:
    previous = np.asarray(previous_weights, dtype=float)
    if previous.shape != desired.shape:
        raise ValueError("previous_weights has the wrong shape.")

eta = float(np.clip(learning_rate, 0.0, 1.0))
updated = (1.0 - eta) * previous + eta * desired
updated[self.passband] /= np.mean(updated[self.passband])
updated[~self.passband] = 0.0

valid_values = smoothed[reliable]
flatness_db = float(
    20.0 * np.log10(
        np.percentile(valid_values, 90) /
        max(np.percentile(valid_values, 10), 1e-15)
    )
)
diagnostics = {
    "reliable_bins": int(np.count_nonzero(reliable)),
    "response_spread_10_to_90_db": flatness_db,
    "median_response": float(target_level),
}
return updated, diagnostics

```

```

def detect_subharmonic_response(
    self,
    received_signal: np.ndarray,
    fundamental_hz: float,
    orders: Sequence[int] = (2, 3),
    bandwidth_hz: float = 20e3,
    reference_band: Optional[Tuple[float, float]] = None,
) -> dict[int, float]:
    """
    Measure received spectral energy around f0/order.

    This detects candidate subharmonic output components; it does not by
    itself prove that a crack generated them. Use hardware and specimen
    controls to exclude source, fixture, and electronics nonlinearities.
    """
    rx = np.asarray(received_signal, dtype=float)
    if rx.shape != (self.N,):
        raise ValueError(f"received_signal must contain {self.N} samples.")
    if not (0.0 < fundamental_hz < self.fs / 2):
        raise ValueError("fundamental_hz must lie below Nyquist.")

    window = np.hanning(self.N)
    spectrum = np.abs(np.fft.rfft(rx * window)) ** 2

    if reference_band is None:
        reference_mask = self.passband
    else:
        a, b = reference_band
        reference_mask = (self.freqs >= a) & (self.freqs <= b)

    noise_reference = float(np.median(spectrum[reference_mask]) + 1e-30)
    results: dict[int, float] = {}

    for order in orders:
        if order < 2:
            raise ValueError("Subharmonic orders must be integers >= 2.")
        center = fundamental_hz / order
        band = np.abs(self.freqs - center) <= bandwidth_hz / 2
        band_power = float(np.sum(spectrum[band]))
        expected_noise = noise_reference * max(1, np.count_nonzero(band))
        results[int(order)] = float(
            10.0 * np.log10((band_power + 1e-30) / expected_noise)
        )
    return results

if __name__ == "__main__":
    np.random.seed(7)
    ndt = UltrasonicNDTProbe()

    tx1, metrics1 = ndt.synthesize(
        target_band=(0.8e6, 1.2e6),
        alpha=1.0,
    )

    # Demonstration channel: frequency-dependent attenuation plus noise.

```

```

X = np.fft.rfft(tx1)
channel = 0.15 + 0.85 * np.exp(
    -0.5 * ((ndt.freqs - 1.05e6) / 0.22e6) ** 2
)
rx1 = np.fft.irfft(X * channel, n=ndt.N)
rx1 += 0.01 * np.random.randn(ndt.N)

weights, diagnostics = ndt.update_feedback(rx1, tx1)
tx2, metrics2 = ndt.synthesize(
    target_band=(0.8e6, 1.2e6),
    alpha=1.0,
    feedback_weights=weights,
)

print("Initial metrics:", metrics1)
print("Updated metrics:", metrics2)
print("Feedback diagnostics:", diagnostics)

```

7. Platform 2: marine acoustic probe

7.1 Interpretation

A logarithmic chirp can distribute time approximately uniformly per octave. Applying a time-varying amplitude proportional to $f(t)^{-\alpha/2}$ creates a power-law-weighted chirp, but the finite-window output spectrum also depends on sweep rate, tapering, source response, propagation, and pulse compression. The weighting therefore remains a design input, not a guarantee of the emitted or received spectrum.

The lower-frequency component below is termed a **companion chirp**. It can improve low-frequency illumination or support cross-band comparison, but it should not be described as a subharmonic generated by the environment.

A 192 kHz sample rate has a 96 kHz Nyquist frequency. A requested 80 kHz upper limit is mathematically representable, but leaves limited reconstruction-filter margin. The implementation defaults to a 45% sampling-rate ceiling and therefore requires an upper frequency no greater than 86.4 kHz; practical hardware may require a wider margin.

7.2 Python implementation

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Tuple

import numpy as np
from scipy.signal import get_window

@dataclass
class SonarProbe:
    composite: np.ndarray
    fundamental: np.ndarray
    companion: np.ndarray
    crest_factor_db: float

class SonarPowerLawChirp:

```

```

"""
Generate a logarithmic acoustic chirp with a power-law amplitude
envelope and an optional lower-frequency companion chirp.
"""

def __init__(
    self,
    sample_rate: float = 192_000.0,
    duration_seconds: float = 0.25,
    nyquist_safety_fraction: float = 0.45,
) -> None:
    if sample_rate <= 0 or duration_seconds <= 0:
        raise ValueError("sample_rate and duration_seconds must be positive.")
    if not (0.1 <= nyquist_safety_fraction < 0.5):
        raise ValueError("nyquist_safety_fraction must lie in [0.1, 0.5).")

    self.fs = float(sample_rate)
    self.duration = float(duration_seconds)
    self.N = int(round(self.fs * self.duration))
    self.t = np.arange(self.N, dtype=float) / self.fs
    self.maximum_frequency = nyquist_safety_fraction * self.fs

def _log_chirp(
    self,
    f_start: float,
    f_end: float,
    alpha: float,
    phase_offset: float = 0.0,
) -> np.ndarray:
    if not (0.0 < f_start < f_end <= self.maximum_frequency):
        raise ValueError(
            f"Require 0 < f_start < f_end <= {self.maximum_frequency:.1f} Hz."
        )

    ratio = f_end / f_start
    k = np.log(ratio) / self.duration

    # Instantaneous frequency and integrated phase for an exponential sweep.
    instantaneous_frequency = f_start * np.exp(k * self.t)
    phase = (
        2.0 * np.pi * f_start * np.expml(k * self.t) / k
        + phase_offset
    )

    amplitude = (
        instantaneous_frequency /
        np.sqrt(f_start * f_end)
    ) ** (-0.5 * alpha)

    taper = get_window(("tukey", 0.12), self.N, fftbins=False)
    return amplitude * np.sin(phase) * taper

def generate(
    self,
    f_start: float,
    f_end: float,

```

```

alpha: float = 1.0,
companion_ratio: float = 0.5,
companion_level_db: float = -18.0,
target_rms: float = 0.18,
peak_limit: float = 0.95,
) -> SonarProbe:
    fundamental = self._log_chirp(
        f_start=f_start,
        f_end=f_end,
        alpha=alpha,
    )

    if companion_ratio <= 0:
        raise ValueError("companion_ratio must be positive.")
    companion_start = f_start * companion_ratio
    companion_end = f_end * companion_ratio

    companion = self._log_chirp(
        f_start=companion_start,
        f_end=companion_end,
        alpha=alpha,
        phase_offset=np.pi / 2,
    )
    companion *= db_to_amplitude(companion_level_db)

    composite = normalize_rms_and_limit_peak(
        fundamental + companion,
        target_rms=target_rms,
        peak_limit=peak_limit,
    )

    # Preserve the components at the same final scale for analysis.
    unscaled_sum = fundamental + companion
    scale = rms(composite) / max(rms(unscaled_sum), 1e-15)
    fundamental *= scale
    companion *= scale

    return SonarProbe(
        composite=composite,
        fundamental=fundamental,
        companion=companion,
        crest_factor_db=crest_factor_db(composite),
    )

if __name__ == "__main__":
    sonar = SonarPowerLawChirp(
        sample_rate=192_000,
        duration_seconds=0.25,
    )

    probe = sonar.generate(
        f_start=20_000,
        f_end=80_000,
        alpha=1.0,
        companion_ratio=0.5,

```

```

        companion_level_db=-18.0,
    )

    print(f"Samples: {probe.composite.size}")
    print(f"RMS: {rms(probe.composite):.4f}")
    print(f"Peak: {np.max(np.abs(probe.composite)):.4f}")
    print(f"Crest factor: {probe.crest_factor_db:.2f} dB")

```

7.3 Recommended measurement separation

When the fundamental and companion bands overlap, attribution becomes difficult. For nonlinear-response studies, prefer one of these designs:

1. transmit the fundamental and companion in separate pings;
2. place the bands in non-overlapping frequency intervals;
3. encode the components with orthogonal sequences and estimate them separately;
4. use repeated amplitude levels and identify nonlinear scaling;
5. compare against source and receiver loopback measurements.

8. Platform 3: LiDAR intensity-modulation multisine

8.1 Interpretation

The following engine creates a periodic, FFT-bin-centered **intensity-modulation multisine** for a laser-driver or electro-optic-modulator input. It does not produce an optical frequency chirp and therefore should not be called FMCW LiDAR.

The output is represented as a normalized electrical command in $[0, 1]$, with a configurable bias and modulation depth. Actual optical power depends on the driver transfer curve, threshold current, thermal behavior, bandwidth, and safety interlocks. A production device must apply calibrated linearization and eye-safety constraints.

8.2 Python implementation

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Sequence

import numpy as np

@dataclass
class LidarWaveform:
    command: np.ndarray
    realized_frequencies_hz: np.ndarray
    crest_factor_db_ac: float
    bias: float
    modulation_depth: float

class LidarIntensityMultisine:
    """
    Cyclic, FFT-bin-centered intensity-modulation multisine.

    Frequencies are quantized to DFT bins so the playback buffer joins
    continuously when repeated.

```

```

"""

def __init__(
    self,
    sample_rate: float = 250e6,
    num_points: int = 4096,
    maximum_frequency_fraction: float = 0.45,
) -> None:
    if sample_rate <= 0 or num_points < 32:
        raise ValueError("sample_rate must be positive and num_points >= 32.")
    if not (0.1 <= maximum_frequency_fraction < 0.5):
        raise ValueError(
            "maximum_frequency_fraction must lie in [0.1, 0.5).")
    )

    self.fs = float(sample_rate)
    self.N = int(num_points)
    self.bin_spacing = self.fs / self.N
    self.maximum_frequency = maximum_frequency_fraction * self.fs
    self.n = np.arange(self.N, dtype=float)

def _quantize_to_bins(
    self,
    requested_frequencies_hz: Sequence[float],
) -> tuple[np.ndarray, np.ndarray]:
    requested = np.asarray(requested_frequencies_hz, dtype=float)
    if requested.ndim != 1 or requested.size == 0:
        raise ValueError("At least one target frequency is required.")
    if np.any(requested <= 0) or np.any(requested > self.maximum_frequency):
        raise ValueError(
            f"Frequencies must lie in (0, {self.maximum_frequency:.1f}] Hz."
        )

    bins = np rint(requested / self.bin_spacing).astype(int)
    if np.any(bins <= 0) or np.any(bins >= self.N // 2):
        raise ValueError("Quantized tones must remain inside positive bins.")

    bins = np.unique(bins)
    realized = bins.astype(float) * self.bin_spacing
    return bins, realized

def generate(
    self,
    target_frequencies_hz: Sequence[float],
    alpha: float = 1.0,
    bias: float = 0.50,
    modulation_depth: float = 0.80,
) -> LidarWaveform:
    if not (0.0 < bias < 1.0):
        raise ValueError("bias must lie strictly between 0 and 1.")
    if not (0.0 < modulation_depth <= 1.0):
        raise ValueError("modulation_depth must lie in (0, 1].")

    bins, realized = self._quantize_to_bins(target_frequencies_hz)
    amplitudes = (
        realized / np.sqrt(realized.min() * realized.max())

```

```

) ** (-0.5 * alpha)
amplitudes /= np.sqrt(np.sum(amplitudes ** 2))

phases = quadratic_multisine_phases(realized.size)
ac = np.zeros(self.N, dtype=float)

for amplitude, bin_index, phase in zip(amplitudes, bins, phases):
    ac += amplitude * np.cos(
        2.0 * np.pi * bin_index * self.n / self.N + phase
    )

ac /= max(np.max(np.abs(ac)), 1e-15)

# The requested modulation depth is peak-to-peak as a fraction of
# full command range. Reduce it automatically if the bias cannot
# support the requested symmetric excursion.
maximum_symmetric_depth = 2.0 * min(bias, 1.0 - bias)
effective_depth = min(modulation_depth, maximum_symmetric_depth)
command = bias + 0.5 * effective_depth * ac
command = np.clip(command, 0.0, 1.0)

return LidarWaveform(
    command=command,
    realized_frequencies_hz=realized,
    crest_factor_db_ac=crest_factor_db(ac),
    bias=bias,
    modulation_depth=effective_depth,
)

if __name__ == "__main__":
    lidar = LidarIntensityMultisine(
        sample_rate=250e6,
        num_points=4096,
    )

    waveform = lidar.generate(
        target_frequencies_hz=[30e6, 60e6, 90e6, 110e6],
        alpha=1.0,
        bias=0.50,
        modulation_depth=0.80,
    )

    print("Realized tones (MHz):",
          waveform.realized_frequencies_hz / 1e6)
    print("Command range:",
          float(waveform.command.min()),
          float(waveform.command.max()))
    print("AC crest factor:",
          f"{waveform.crest_factor_db_ac:.2f} dB")

```

9. Fully executable consolidated implementation

The downloadable source document includes the three modules above. For direct execution, place the shared utility functions before the platform classes, install:

```
python -m pip install numpy scipy
```

The HTML edition includes a single copy-pasteable code appendix without the typesetting placeholder.

10. Validation strategy

A waveform generator can execute without exceptions and still fail experimentally. Validation should occur in layers.

10.1 Numerical validation

- verify all tones lie below the configured frequency ceiling;
- verify cyclic endpoint continuity for repeated buffers;
- report RMS, peak, and crest factor;
- inspect the actual FFT rather than assuming the requested spectrum;
- test zero, empty, and invalid inputs;
- confirm that feedback weights remain finite and bounded;
- confirm that the regularized inverse respects its gain ceiling.

10.2 Electrical and electroacoustic validation

- measure DAC and amplifier output into a safe load;
- verify anti-imaging and anti-alias filtering;
- measure the complex transmitter response with a calibrated receiver;
- test for clipping, slew-rate limiting, intermodulation, and thermal drift;
- repeat at several drive levels;
- preserve a loopback channel where possible.

10.3 NDT nonlinear validation

- compare damaged and undamaged reference specimens;
- measure fixture, couplant, electronics, and transducer nonlinearities;
- inspect amplitude-dependent scaling of candidate harmonics and subharmonics;
- use coherent averaging and confidence intervals;
- avoid interpreting one spectral line as defect proof without controls.

10.4 Sonar validation

- use source levels and frequency bands permitted for the transducer and operating environment;
- account for absorption, beam pattern, reverberation, bubble clouds, and platform motion;
- perform pulse compression with the transmitted reference;
- separate companion-band returns from nonlinear products.

10.5 LiDAR validation

- characterize the laser-driver current-to-optical-power curve;
- maintain positive bias above any required threshold;
- avoid frequencies beyond the analog modulator bandwidth;
- comply with applicable laser safety classification and interlocks;
- distinguish intensity modulation from optical-frequency FMCW ranging.

11. Demonstration figures and performance interpretation

The following figures use a controlled simulation to illustrate the intended behavior. They do not represent measured hardware performance. Their purpose is to show how an adaptive loop can redistribute effort and how the proposed metrics should be reported.

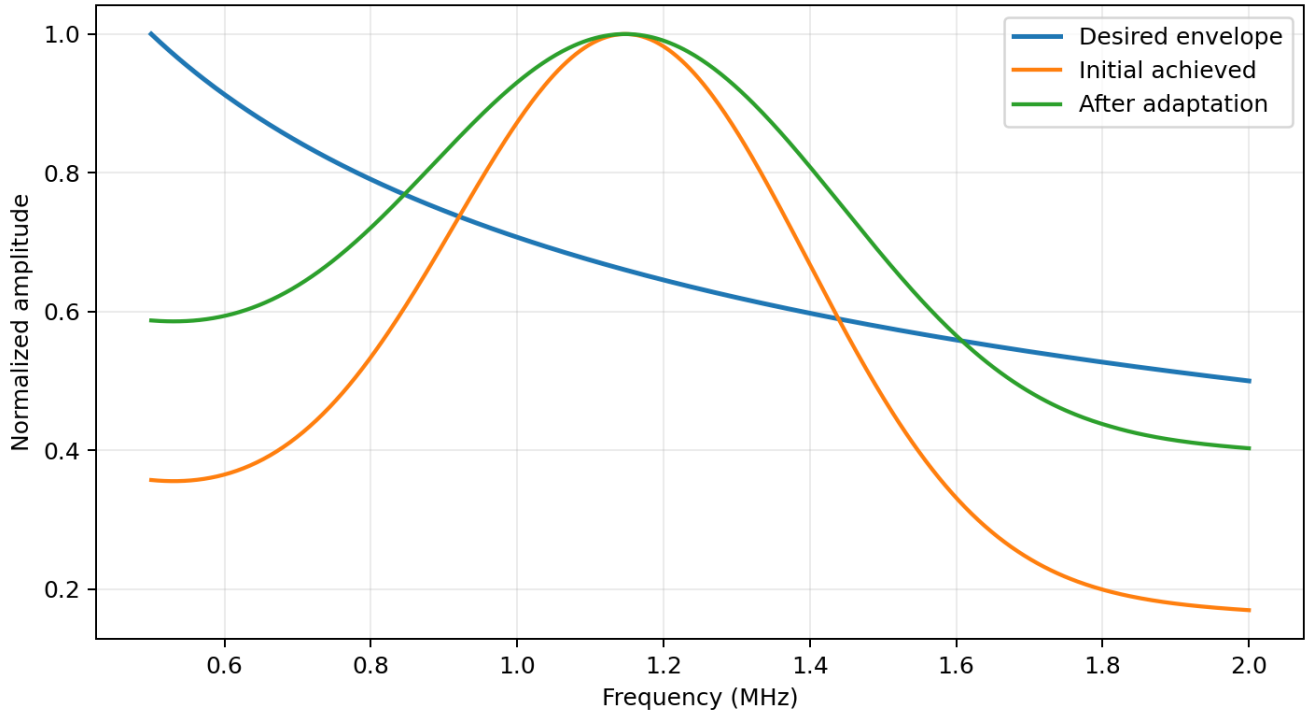


Figure 1: Desired and achieved spectral allocation across adaptive cycles.

Figure 1. Desired and achieved spectral allocation in a simulated frequency-dependent channel. The adaptive cycle increases weight in attenuated but reliable regions while remaining bounded by the configured limits.

Figure 2. Example feedback-weight evolution. Early cycles make larger corrections; later cycles settle as the measured response approaches the selected objective.

Figure 3. Illustrative crest-factor comparison for equal-amplitude tones. Quadratic phases often lower crest factor relative to aligned phases, although the result depends on tone placement and amplitudes.

11.1 Interpreting improvement

A useful report should distinguish waveform quality from sensing quality. Crest factor, RMS level, spectral error, and convergence speed describe the probe. Detection probability, defect SNR, classification accuracy, posterior uncertainty, and false-alarm rate describe the sensing task. A successful adaptive waveform does not automatically establish better diagnosis; the final comparison must use task-level metrics on controlled data.

For outlier-oriented deployment, the most relevant comparison may be hierarchical:

- cost and accuracy of routine screening alone;
- cost and accuracy of screening followed by fixed diagnostic rescanning;
- cost and accuracy of screening followed by adaptive attention.

This comparison makes the additional up-front cost explicit and tests whether focused follow-up produces enough information to justify it.

12. Conclusions

The adaptive power-law probe loop defines a calibrated and testable form of sensor attention:

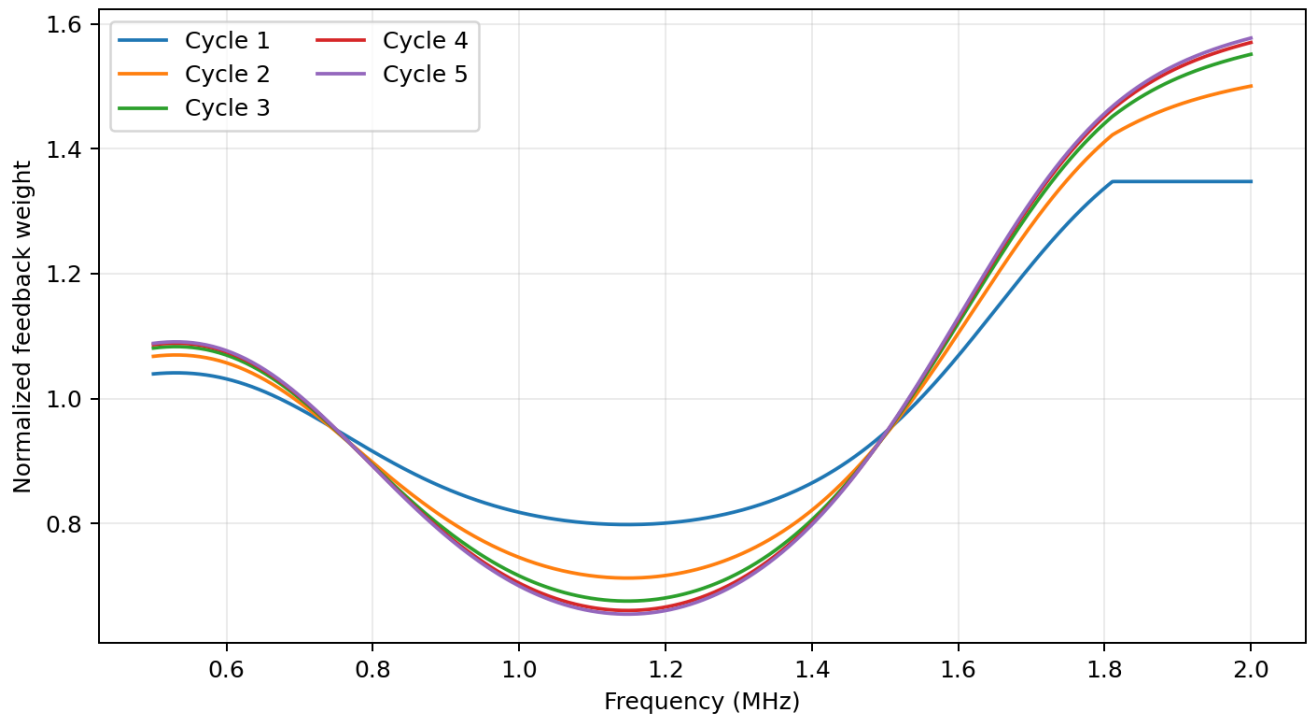


Figure 2: Evolution of feedback weights over five cycles.

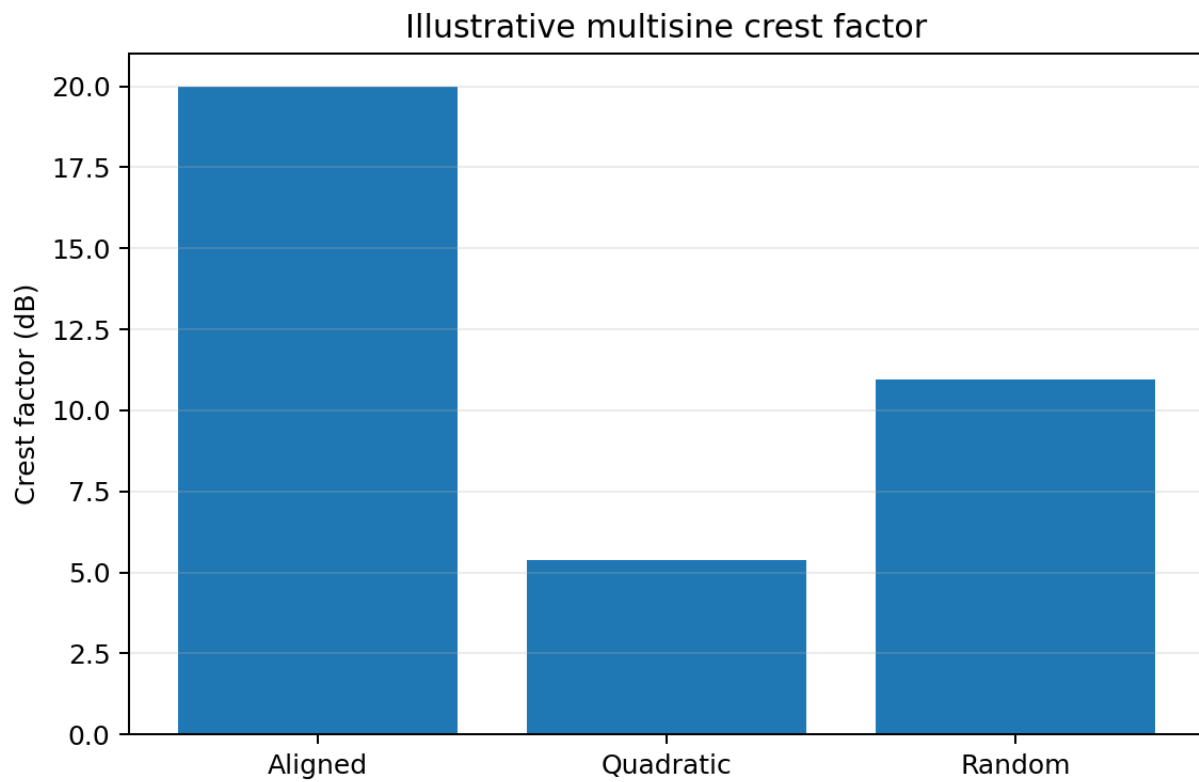


Figure 3: Crest-factor comparison among phase strategies.

- power-law weighting forms a tunable family rather than a universal law;
- a $1/f$ power spectrum requires a $1/\sqrt{f}$ Fourier-amplitude envelope;
- regularized complex inversion and gain limiting replace direct division by a transducer response;
- quadratic multisine phases can reduce crest factor but require measurement and do not guarantee an optimum;
- bounded, reliability-masked feedback replaces raw response multiplication;
- transmitted lower-frequency companion bands must not be confused with specimen-generated subharmonics;
- the LiDAR example implements intensity modulation, not optical FMCW;
- hardware safety, calibration, and controlled experiments remain necessary before physical conclusions can be drawn.

Within those limits, the framework supplies a coherent scaffold for comparing probe spectra, adapting excitation to measured responses, and testing whether selective allocation improves a specific sensing objective. Its strongest intended role lies not in replacing inexpensive routine scans, but in directing additional attention toward ambiguous, anomalous, or diagnostically important cases where feedback can justify the added sensing cost.

References

1. S. Boyd, "Multitone Signals with Low Crest Factor," *IEEE Transactions on Circuits and Systems*, 33(10), 1018-1022, 1986.
2. Literature on regularized inverse filtering and electroacoustic response correction, including Tikhonov-style inverse filters and gain-constrained source equalization.
3. Reviews and experimental studies of nonlinear ultrasonic testing showing that harmonics and subharmonics can arise at contacting or partially closed cracks, while also requiring controls for source and boundary nonlinearities.
4. Woods Hole Oceanographic Institution, field description of chirp sub-bottom profiling, including the penetration-resolution tradeoff and representative 4-24 kHz operation.
5. Standard sampling-theorem, FFT, and multisine system-identification principles used throughout the implementation.